AI APPLICATIONS | HANDS-ON LABORATORY

Part I: Local CrewAI Deployment and Flask Web Application

From a CrewAI Studio workflow to a locally executable, document-driven AI application

Audience: Undergraduate students in AI Applications

Format: Detailed, instructor-guided laboratory with runnable code and checkpoints

Suggested duration: 3-5 hours, depending on environment setup and LLM runtime

Companion: Part II - CrewAI Cloud Deployment, REST API Authentication, and Flask Integration

Case study: Digital Policy Change Request System - transform meeting transcripts into a Business Requirements Document (BRD).

Instructor note. This guide reconstructs the successful local deployment workflow and key troubleshooting lessons from the course development exercise. Exact export layouts, package versions, and CrewAI Studio controls may differ across student accounts and product releases. Students should inspect their actual exported project rather than assume all exports are identical.

1. Learning objectives and deliverable

Students will export an existing CrewAI Studio workflow, install its Python dependencies, execute the workflow locally, expose it through Flask, and build a browser interface that accepts a Word meeting transcript and displays the resulting BRD.

• Explain the distinction between a declarative workflow definition and executable Python runtime.

• Set up a reproducible Python project with uv and environment variables.

• Execute a CrewAI flow locally using a transcript input.

• Build a Flask API that returns structured JSON responses.

• Read DOCX paragraphs and tables using python-docx.

• Connect a browser file-upload interface to the Flask endpoint.

• Test and debug the complete workflow while protecting credentials and sensitive data.

Expected deliverable

A local Flask web application accessible at http://127.0.0.1:5000/. A user selects a .docx meeting transcript, clicks Generate BRD, waits for the CrewAI agents to finish, and views or downloads the generated BRD as text.

What students need

Item	Purpose
CrewAI Studio workflow	Existing agent/task/flow design that generates a BRD
Python 3.13-compatible environment	Local Python execution; use the version compatible with the export
uv	Dependency and virtual environment management
VS Code and terminal	Edit code and run the application
LLM provider API key	Authenticate the model used by the workflow
Test meeting transcript	Prefer synthetic, nonconfidential DOCX content

2. Architecture and vocabulary

Studio design: agents, tasks, conditions, and outputs are configured in CrewAI Studio. Export: a ZIP containing a project definition and supporting configuration. Local runtime: CrewAI packages interpret the exported declarative workflow. Flask: an HTTP layer and browser interface around the local execution function.

Execution path: Browser -> Flask /upload -> python-docx extraction -> CrewAI declarative flow -> LLM API -> generated BRD -> Flask JSON -> Browser.

The agents execute on the local machine, but a hosted LLM still requires internet access and may incur usage charges. Local deployment does not necessarily mean the language model runs locally.

3. Export the CrewAI Studio project

- Open the working BRD workflow in CrewAI Studio.
- Run a test inside Studio and verify that the output is a usable BRD.
- Locate the project export or Download control in the Studio interface.
- Download the ZIP and extract it into a new project folder.
- Open that folder in VS Code and inspect its contents.

Understand the exported files

```
Digital-Policy-Change-Request-System/
  pyproject.toml
  uv.lock                     # if included by the export
  src/
    digital_policy_change_request_system/
      flow.json
  .env                        # create locally; do not publish
  app.py                      # created later in this lab
  templates/
    index.html                # created later in this lab
```

A Studio export may contain substantial JSON/YAML configuration and relatively little handwritten Python. That is expected for a declarative CrewAI workflow: the installed runtime loads and executes the definitions. It is not evidence that the export is missing all functionality.

Checkpoint: Find flow.json and identify the input variable named transcript. Locate at least one agent, one task, and the final BRD-producing step in the exported definitions.

4. Prepare the local Python environment

Step 4.1 - Verify tools

```
python --version
uv --version
```

The original exercise used Python 3.13 and an uv-managed environment. Students should follow the compatible Python requirement specified in their project configuration rather than install arbitrary versions.

Step 4.2 - Install project dependencies

In the VS Code terminal, change to the extracted project root (the folder containing pyproject.toml).

```
uv sync --python 3.13
```

uv creates or updates .venv and installs dependencies. In the reference exercise, a first synchronization installed approximately 159 packages; counts and timing vary.

Step 4.3 - Verify imports

```
uv run python -c "import crewai, flask, docx; print('Imports OK')"
```

If Flask or python-docx is missing from the project requirements, add it explicitly:

```
uv add flask python-docx python-dotenv requests
```

The requests library is used in Part II; including it now simplifies the transition. Students may omit it from Part I if desired.

Checkpoint: Show successful Python and uv version output, a completed uv sync, and an Imports OK message.

5. Configure credentials and inspect the workflow

Step 5.1 - Create .env

```
OPENAI_API_KEY=YOUR_OWN_API_KEY
```

Use the credential required by the model configured in your exported flow. For the reference workflow, the LLM was an OpenAI model. Never paste a real key into code, screenshots, GitHub, or submitted reports.

Step 5.2 - Protect secrets

```
.env
.env.*
!.env.example
.venv/
__pycache__/
*.pyc
.crewai/
output/
```

Create or update .gitignore with these entries. If a key was ever committed, removing it from the latest file is not sufficient: revoke/rotate the credential and address the repository history.

Step 5.3 - Verify the workflow path

```
src/digital_policy_change_request_system/flow.json
```

The directory name may differ in a student export. Adjust the Python path accordingly. The file must exist before attempting local execution.

Step 5.4 - Common .env encoding problem

In the reference exercise, an environment-file encoding issue was resolved by saving .env as UTF-8 without a byte-order mark (BOM). If an API key appears to be missing even though it is present, check the filename, working directory, variable spelling, and file encoding.

6. Experiment 1 - Execute the workflow locally

First verify that the CrewAI workflow itself runs, independently of Flask. If the export supports a command-line entry point, the reference exercise used:

```
uv run crewai run
```

The command may prompt for inputs or follow project-specific configuration. Students should consult their exported project README or CLI help for the exact invocation.

A direct Python execution script

Create test_local_flow.py in the project root:

```
from pathlib import Path
from dotenv import load_dotenv
from crewai_cli.run_declarative_flow import load_declarative_flow

ROOT = Path(__file__).resolve().parent
load_dotenv(ROOT / ".env")
FLOW_PATH = ROOT / "src" / "digital_policy_change_request_system" / "flow.json"

transcript = (
    "Customers need to request policy changes online. "
    "Staff review and approve requests. "
    "The system must maintain an audit trail."
)

flow = load_declarative_flow(FLOW_PATH)
result = flow.kickoff(inputs={"transcript": transcript})
print(str(result))

uv run python test_local_flow.py
```

This approach depends on the declarative-flow loader available in the exported project environment. If the import fails, inspect the installed CrewAI CLI version and the export instructions; do not assume the loader API is identical in every release.

What to inspect

• Did the flow start and reach its final task?

• Did the output include a BRD with meaningful requirements?

• Were there authentication, model, or rate-limit errors?

• How long did the execution take and what usage information was reported?

In the reference workflow, successful executions typically took several minutes and had small per-run model charges. Those observations are examples, not pricing guarantees.

Checkpoint: Save a short, synthetic transcript and a sample BRD excerpt. Confirm that the local workflow can run before introducing Flask.

7. Experiment 2 - Build a Flask REST interface

We first create a simple JSON endpoint. It accepts a transcript and returns the generated BRD as JSON.

Step 7.1 - Create app.py

```
from pathlib import Path
from flask import Flask, request, jsonify
from dotenv import load_dotenv
from crewai_cli.run_declarative_flow import load_declarative_flow

app = Flask(__name__)
ROOT = Path(__file__).resolve().parent
load_dotenv(ROOT / ".env")
FLOW_PATH = ROOT / "src" / "digital_policy_change_request_system" / "flow.json"


def execute_crewai(transcript):
    flow = load_declarative_flow(FLOW_PATH)
    result = flow.kickoff(inputs={"transcript": transcript})
    return str(result)


@app.post("/run")
def run_workflow():
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        return jsonify({"error": "Expected JSON input"}), 400
    transcript = data.get("transcript")
    if not isinstance(transcript, str) or not transcript.strip():
        return jsonify({"error": "Transcript is required"}), 400
    try:
        result = execute_crewai(transcript)
        return jsonify({"status": "success", "result": result})
    except Exception:
        app.logger.exception("CrewAI execution failed")
        return jsonify({"status": "error", "message": "Execution failed"}), 500


if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=False)
```

Step 7.2 - Run Flask

```
uv run python app.py
```

Open another terminal to test POST /run. PowerShell users may use Invoke-RestMethod:

```
$body = @{ transcript = "Customers submit policy changes online." } | ConvertTo-Json
Invoke-RestMethod -Uri http://127.0.0.1:5000/run -Method Post `
  -ContentType "application/json" -Body $body
```

PowerShell backticks indicate line continuation; enter the command exactly as shown or use a REST client such as Postman. Each POST may initiate a paid model execution.

Checkpoint: Confirm that the endpoint returns JSON with status=success and a nonempty result.

8. Experiment 3 - Accept a Word meeting transcript

The next feature lets a user upload a DOCX meeting transcript instead of copying its contents into JSON. The python-docx library reads Word files without requiring Microsoft Office to be installed.

Step 8.1 - Add imports and upload limit

```
from io import BytesIO
from docx import Document
from flask import render_template

app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024
```

Place these imports with the other imports in app.py and configure the size limit after Flask initialization.

Step 8.2 - Extract paragraphs and tables

```
def extract_docx_text(file_bytes):
    doc = Document(BytesIO(file_bytes))
    parts = []
    for block in doc.iter_inner_content():
        if hasattr(block, "rows"):
            for row in block.rows:
                cells = [cell.text.strip() for cell in row.cells]
                parts.append(" | ".join(cells))
        elif hasattr(block, "text") and block.text.strip():
            parts.append(block.text)
    return "
".join(parts)
```

This example preserves the reading order of common paragraphs and tables. It does not extract text embedded only in images, and complex DOCX layouts may require additional handling.

Step 8.3 - Create the upload endpoint

```
@app.post("/upload")
def upload_document():
    if "file" not in request.files:
        return jsonify({"error": "No file uploaded"}), 400

    uploaded = request.files["file"]
    if not uploaded.filename:
        return jsonify({"error": "No file selected"}), 400
    if not uploaded.filename.lower().endswith(".docx"):
        return jsonify({"error": "Only .docx files supported"}), 400

    try:
        transcript = extract_docx_text(uploaded.read())
    except Exception:
        app.logger.exception("DOCX parsing failed")
        return jsonify({"error": "Could not read Word document"}), 400

    if not transcript.strip():
        return jsonify({"error": "Document has no extractable text"}), 400

    try:
        result = execute_crewai(transcript)
        return jsonify({
            "status": "success",
            "filename": uploaded.filename,
            "transcript_length": len(transcript),
            "result": result
        })
    except Exception:
        app.logger.exception("CrewAI execution failed")
        return jsonify({"status": "error", "message": "Execution failed"}), 500
```

For a production application, validate uploaded file content and access permissions more rigorously. An extension check alone does not establish that a file is safe or valid.

Checkpoint: Upload a synthetic .docx and verify that Flask extracts its text and returns a BRD. This does not require Microsoft Word.

9. Experiment 4 - Create the browser interface

Create a templates folder and save the following page as templates/index.html. Flask automatically looks for templates in this directory.

Step 9.1 - Add a homepage route

```
@app.get("/")
def home():
    return render_template("index.html")
```

Place this route above the final if __name__ == "__main__" block in app.py.

Step 9.2 - Create templates/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>CrewAI BRD Generator</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 900px;
           margin: 40px auto; padding: 0 18px; color: #17324b; }
    button { padding: 10px 18px; cursor: pointer; }
    pre { white-space: pre-wrap; background: #eef3f8;
          padding: 16px; border-radius: 8px; }
  </style>
</head>
<body>
  <h1>CrewAI Business Requirements Document Generator</h1>
  <p>Upload a Word meeting transcript (.docx).</p>
  <form id="form">
    <input type="file" name="file" accept=".docx" required>
    <button type="submit">Generate BRD</button>
  </form>
  <p id="message"></p>
  <pre id="result"></pre>
  <button id="download" hidden>Download BRD (.txt)</button>
  <script>
    const form = document.getElementById('form');
    const message = document.getElementById('message');
    const output = document.getElementById('result');
    const download = document.getElementById('download');
    let lastBRD = '';

    form.addEventListener('submit', async (event) => {
      event.preventDefault();
      message.textContent = 'Generating BRD. Please wait...';
      output.textContent = '';
      download.hidden = true;
      try {
        const response = await fetch('/upload', {
          method: 'POST', body: new FormData(form)
        });
        const data = await response.json();
        if (!response.ok || data.status !== 'success') {
          throw new Error(data.error || data.message || 'Request failed');
        }
        lastBRD = data.result;
        output.textContent = lastBRD;
        download.hidden = false;
        message.textContent = 'BRD generated successfully.';
      } catch (error) {
        message.textContent = 'Error: ' + error.message;
      }
    });

    download.addEventListener('click', () => {
      const blob = new Blob([lastBRD], {type: 'text/plain'});
      const url = URL.createObjectURL(blob);
      const a = document.createElement('a');
      a.href = url; a.download = 'generated_brd.txt'; a.click();
      URL.revokeObjectURL(url);
    });
  </script>
</body>
</html>
```

The browser submits multipart/form-data to /upload, receives JSON, and displays the result as plain text. Using textContent avoids interpreting model output as HTML. A later enhancement can render Markdown safely with a suitable library and sanitization.

Step 9.3 - End-to-end test

```
uv run python app.py
```

Open http://127.0.0.1:5000/, select a DOCX transcript, and click Generate BRD. Confirm that the generated text appears and that the download button saves it as a TXT file.

Checkpoint: Demonstrate the full browser -> Flask -> local CrewAI -> BRD -> browser flow.

10. Troubleshooting and lessons from the reference exercise

Symptom	Likely cause / action
uv sync fails	Check pyproject.toml, Python compatibility, package resolution, and network access.
API key not detected	Check .env path, variable name, and UTF-8 encoding without BOM.
No flow.json found	Inspect the exported folder and update FLOW_PATH.
Import error for declarative loader	Check the installed CrewAI CLI version and the export documentation.
Agents fail or stall	Review terminal logs, LLM model access, rate limits, and input quality.
DOCX cannot be read	Confirm a valid DOCX, not an old .doc or renamed file.
No extractable text	The document may be image-only or contain unsupported content.
Browser shows 404	Confirm the homepage route and templates/index.html location.
Browser waits several minutes	Local workflow is synchronous; inspect Flask logs and allow sufficient time.
BRD text looks like Markdown	Current interface displays raw Markdown; formatting is an enhancement.

A systematic debugging sequence

- Verify Python, uv, and imports.
- Verify the model credential and flow.json path.
- Run the CrewAI flow without Flask.
- Test Flask /run using JSON.
- Test DOCX extraction and /upload.
- Finally, test the browser interface.

This order isolates failures. A working browser cannot compensate for a broken workflow, and a working workflow cannot compensate for a broken HTTP endpoint.

11. Security, responsible use, and limitations

• Use synthetic or authorized transcripts; avoid uploading confidential stakeholder data without permission.

• Never commit .env or API keys. Check Git staging before publication.

• Limit upload sizes and do not expose the Flask development server directly to the public internet.

• Keep debug=False for normal use; production deployment requires a production-grade server and additional controls.

• Record model version, workflow version, input, and evaluation criteria when comparing BRD quality.

• AI-generated requirements can contain omissions, invented assumptions, or unsupported approval claims. Human review is required.

In particular, a BRD heading that says APPROVED is not proof that an authorized stakeholder approved the requirements. Approval must be established by the real project governance process.

12. Student exercises and deliverables

Required exercises

• Explain the role of flow.json and why the exported ZIP may contain few handwritten Python files.

• Demonstrate a successful local flow execution using a short synthetic transcript.

• Submit a DOCX transcript through the browser and save the generated BRD.

• Identify at least three ambiguities or missing requirements in the BRD and propose improvements.

• Capture one useful troubleshooting lesson and describe the diagnostic sequence.

Optional extensions

• Add a clear execution-status indicator and elapsed-time display.

• Provide Markdown preview or DOCX export with appropriate safety checks.

• Add transcript validation and structured error messages.

• Create a local/cloud execution mode switch as preparation for Part II.

Submission checklist

• Project source files, excluding credentials and confidential data.

• Screenshot of a successful local workflow run.

• Screenshot of the browser displaying the generated BRD.

• A sample synthetic transcript and BRD excerpt.

• A 1-2 page reflection on architecture, testing, and BRD quality.

Suggested grading rubric

Criterion	Points
Environment and dependency setup	15
Local CrewAI execution	25
Flask REST API and error handling	20
DOCX upload and browser interface	25
Testing, reflection, and security	15
TOTAL	100

13. Transition to Part II - CrewAI Cloud

Part I establishes a functioning local baseline. In Part II, students will deploy the same workflow to CrewAI Cloud, obtain a deployment-specific Bearer Token, test the /healthcheck, /inputs, /kickoff, and /status endpoints, and modify Flask to support cloud execution.

The most important architectural transition is replacing direct local flow execution with an authenticated REST API call, while retaining the browser and document-upload experience.

Final checkpoint: Before starting Part II, confirm that the local Flask application runs, the document upload succeeds, and the generated BRD is displayed correctly.

End of Laboratory Guide - Part I